Recommender Systems 2

Introduction to Data Mining

Seoul National University

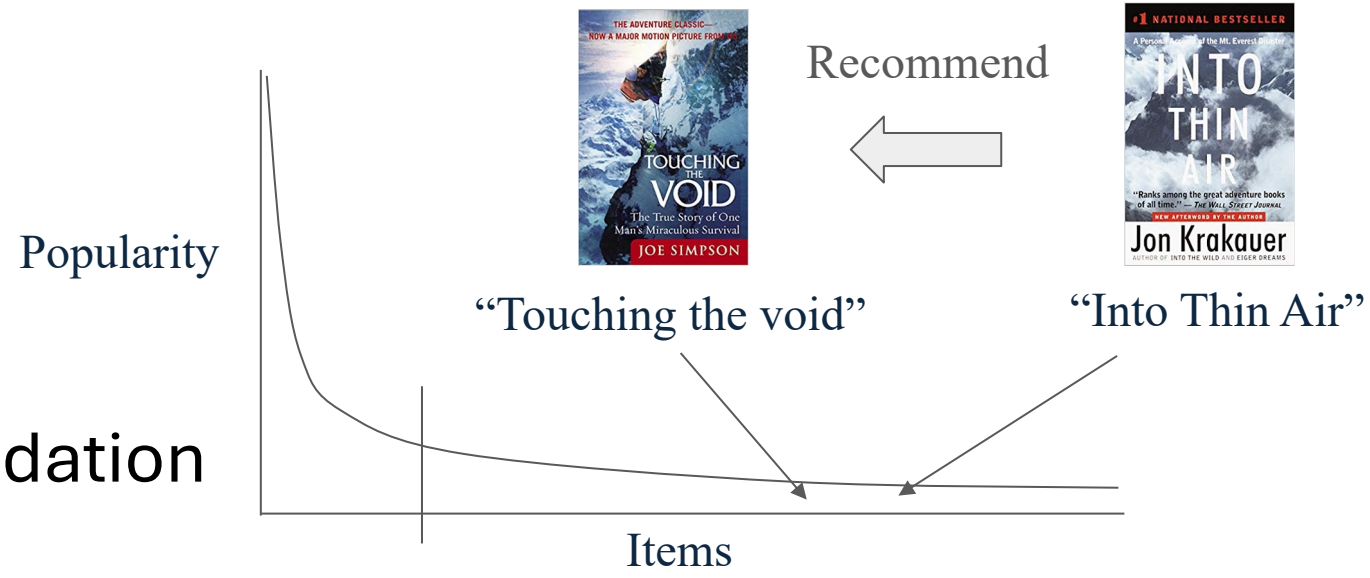
Spring 2026

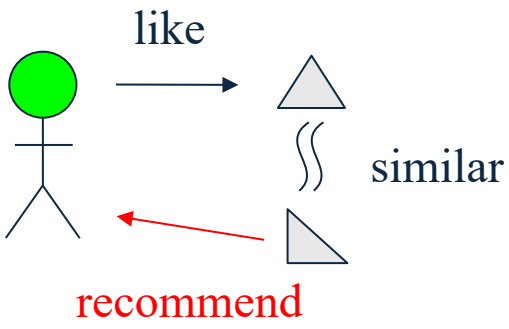
Announcements (Same as Before)

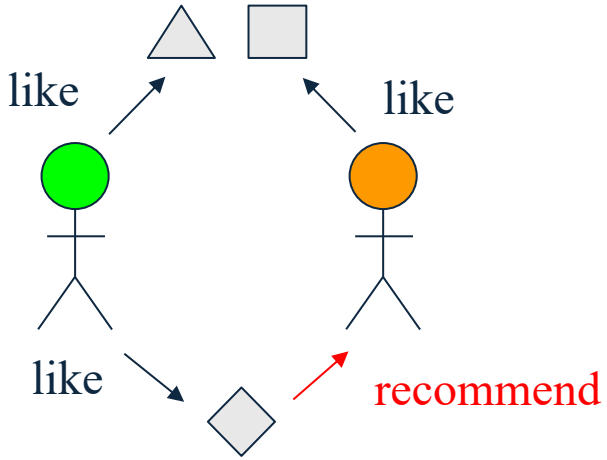
- **Assignments:**
  - HW2 (deadline: April 30)
- **Midterm:**
  - Announcement is uploaded (see eTL).

Recap

- Recommender Systems
- Content-based Recommendation
- Collaborative Filtering
- The Netflix Challenge







Outline

1. **<u>UV Decomposition</u>**
2. UV Decomposition as Machine Learning
3. Variants of UV Decomposition
4. Dealing with Implicit Feedback

Goal of Recommender Systems

- Recommendation is to fill in the blank in the utility matrix R.
- The core operation is to get **user** and **item representations**.
  - Content-based approach creates user/item profiles.
  - Collaborative filtering uses the rows and columns of R.

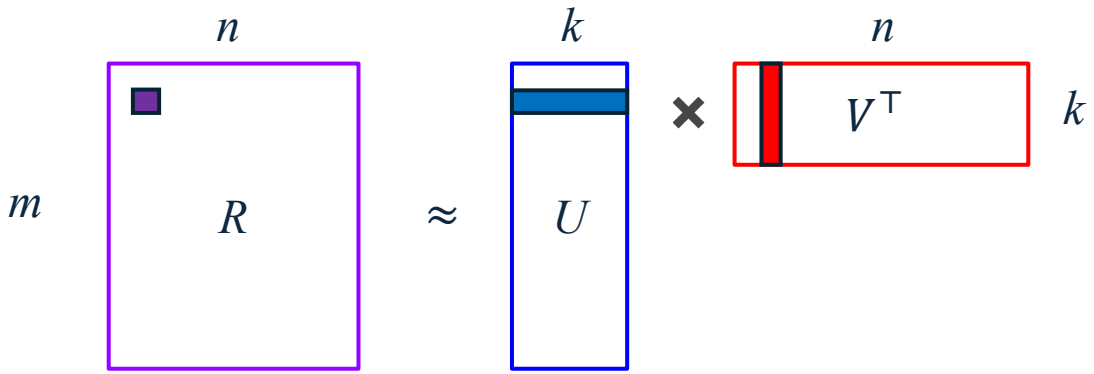
	HP1	HP2	HP3	TW	SW1	SW2	SW3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Latent Factor Models

- We'll learn **latent factor models,** which assume:
  - There are **latent factors** that represent users and items well.
  - Such latent factors can be extracted from the utility matrix.
- Latent factor models are basically **collaborative filtering.**
  - Focus on user-item interactions without using external data.
  - People mostly refer to latent factor models when mentioning CF.

Latent Factor Models: Idea

- **Idea:** Consider the matrix R as the product of **factor matrices**.
  - Latent factors represent underlying concepts/topics.
  - E.g., users react to certain genres, famous actors, or directors.
- **UV decomposition** decomposes R into U and V.
  - Each user and movie is summarized as a low-dimensional vector.



UV Decomposition

- **Given** an $m \times n$ utility matrix R (i.e., m users and n items).
- **Find** an $m \times k$ matrix U and $n \times k$ matrix V such that:
  - $UV^\top$ closely approximates M for the **non-blank entries**.
- **Predict** the blank entries in R using the elements of $UV^\top$.
  - Compute $\hat{r}_{xi} = u_x^\top v_i$ to estimate r_{xi}.

$$\begin{bmatrix} 5 & 2 & 4 & 4 & 3 \\ 3 & 1 & 2 & 4 & 1 \\ 2 & & 3 & 1 & 4 \\ 2 & 5 & 4 & 3 & 5 \\ 4 & 4 & 5 & 4 & \end{bmatrix} = \begin{bmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \\ u_{31} & u_{32} \\ u_{41} & u_{42} \\ u_{51} & u_{52} \end{bmatrix} \times \begin{bmatrix} v_{11} & v_{12} & v_{13} & v_{14} & v_{15} \\ v_{21} & v_{22} & v_{23} & v_{24} & v_{25} \end{bmatrix}$$

R $\qquad$ U $\qquad$ $V^\top$

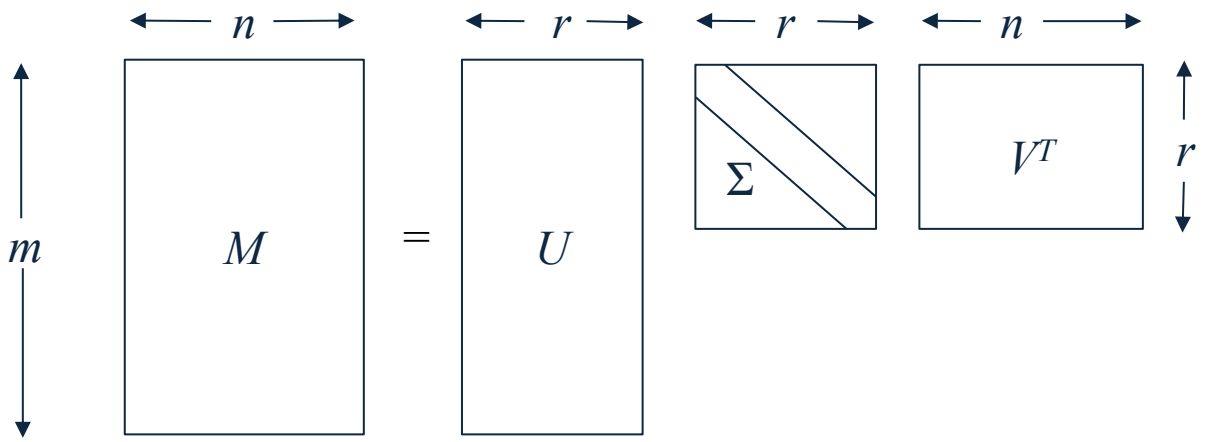
Error Function

- **Root-mean-square error (RMSE)** measures the difference.
  - Let E be the set of non-blank entries.
  - $\mathrm{RMSE}(\hat{R}, R) = \mathrm{sqrt}\left(\frac{1}{|E|}\sum_{(x,i)\in E}(\hat{r}_{xi} - r_{xi})^2\right)$.
- **Sum of squared errors (SSE)** is similar but more convenient.
  - $\mathrm{SSE}(\hat{R}, R) = |E| \cdot \mathrm{RMSE}^2(\hat{R}, R)$.

$$\mathrm{RMSE}\left(\underbrace{\begin{bmatrix} 5 & 2 \\ 3 & \end{bmatrix}}_{R}, \underbrace{\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}}_{UV^\top}\right) = \sqrt{\frac{(5-2)^2+(2-2)^2+(3-2)^2}{3}} = 1.826$$

UV Decomposition vs. SVD

- SVD decomposes R into a product of three matrices.
- **Similarities between UV decomposition and SVD:**
  - Both can choose the number r of latent factors.
  - $U\Sigma$ and V^{T} from SVD can work as factor matrices.



Differences from SVD

- UV decomposition is mainly an **empirical approach.**
  - U and V need not to be orthonormal matrices.
  - Relies on stochastic optimization, not a closed-form solution.
- UV decomposition seeks **generalization** to unseen data.
  - UV ignores the missing entries, not treating them as zero.
  - UV minimizes RMSE on only the training portion.
  - Larger k is not necessarily better (may increase test RMSE).

Outline

1. UV Decomposition
2. **<u>UV Decomposition as Machine Learning</u>**
3. Variants of UV Decomposition
4. Dealing with Implicit Feedback

What is Machine Learning?

- Given data D, we want a function f to solve a problem P.
- **Traditional data mining:** Engineers design f on their own.
  - E.g., A-priori is designed to find frequent itemsets.
- **Machine learning (ML):** Engineers "learn" f from D.
  - Requires **training**, which is computationally expensive.
  - Not necessary if an existing algorithm is already good for P.

UV Decomposition as Machine Learning

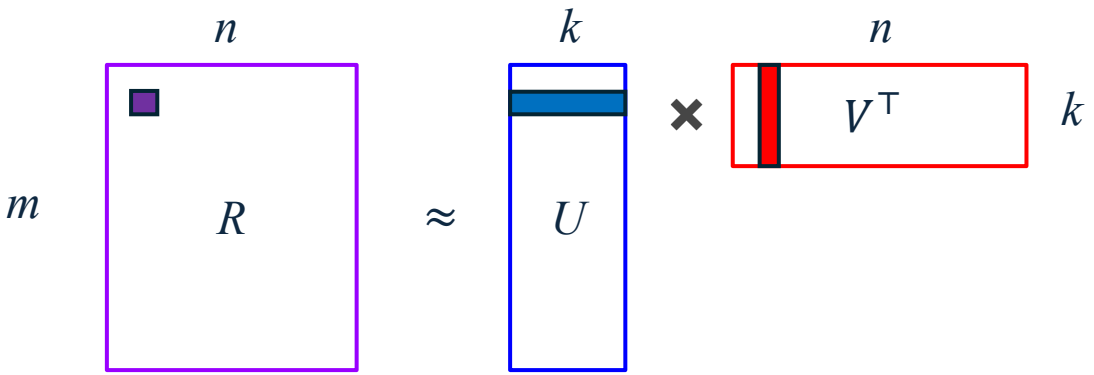
- In traditional CF, we **manage the rule** to recommend items.
  - Based on a similarity measure between users/items.
- In UV decomposition, we just **learn** factor matrices from R.
  - We expect them to be better than the raw rows/columns.
  - We don't know exactly how; no closed-form solution exists.
  - We just want them to minimize the (training) RMSE.

Objective Function

- **Goal:** Find matrices U and V such that:

$$U^*, V^* = \text{argmin}_{U,V}\, J(R, U, V)\,.$$

  - $J(R, U, V) = \sum_{(x,i)\in E}\left(r_{xi}^\top - u_x v_i\right)^2$ measures SSE on training data E.



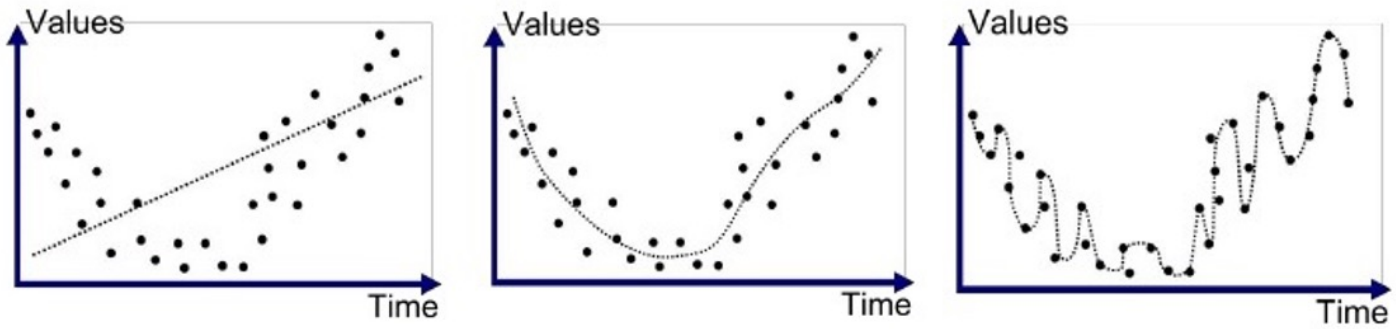
- We call U, V **parameters**.
- Other non-learnable choices are called **hyperparameters**.

Mismatch in the Objective Function

- Our (true) goal is to minimize SSE for **unseen test data**.
- Increasing k *almost always decreases* the value of $J(\cdot)$.
  - Since the error decreases with more factors.
- However, SSE on test data can begin to rise with large k.
  - Since our objective function only considers training data.

Overfitting

- This is a classical example of **overfitting:**
  - Model starts fitting noise with too many free parameters.
  - Model is not generalizing well to unseen test data.
- We should carefully control the model complexity.
  - E.g., the number of clusters in k-means.



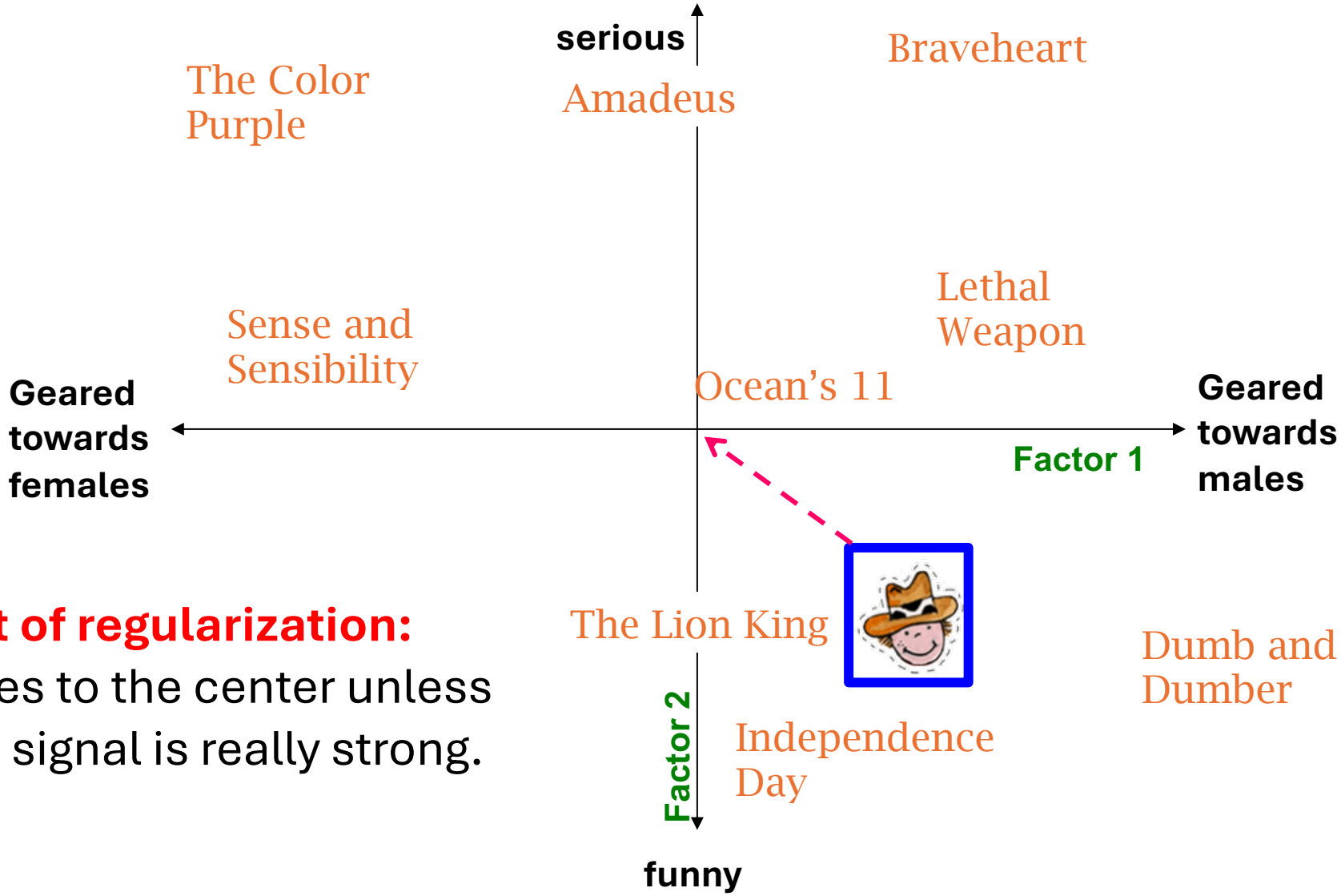
Source: Medium

Regularization

- **Regularization** is a possible way to prevent overfitting:
  - Allows a rich model when there is sufficient data.
  - Shrinks the model aggressively where data is scarce.
- The new objective function with regularization is

$$J(R,U,V) = \sum_{(x,i)\in E} (r_{xi} - u_x v_i)^2 + \underbrace{\lambda_1 \sum_x \|u_x\|_2^2 + \lambda_2 \sum_i \|v_i\|_2^2}_{\text{Limit the lengths of } u_x \text{ and } v_i}$$

  - λ_1 and λ_2 are hyperparameters.



Effect of regularization:

- Goes to the center unless the signal is really strong.

Validation Data

- Introducing **validation data** is also important.
  - Split the training data into (new) training and validation data.
  - Check the performance on validation data during training.
  - Use the validation performance as a proxy of the test performance.

$$\begin{bmatrix} 5 & & 4 & & 3 & 2 \\ 3 & 1 & 2 & 4 & & 1 \\ 2 & & 3 & ? & ? & \\ 2 & 5 & 4 & & ? & \\ 4 & & 5 & ? & & \end{bmatrix}$$

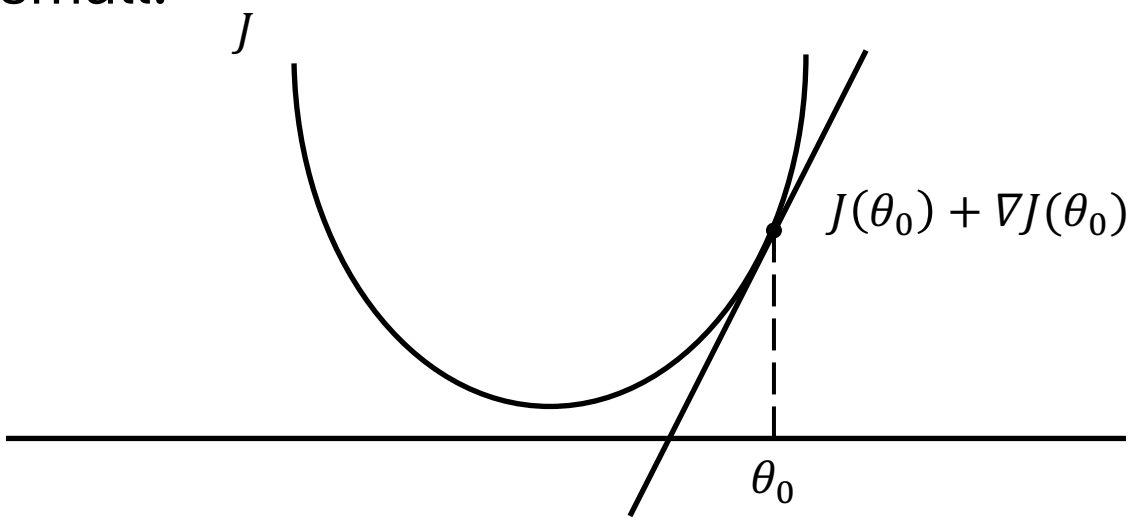
Validation dataset

Training dataset

Test dataset

Gradient Descent

- **Gradient descent (GD)** finds parameters θ^* to minimize $J(\theta)$.
  - Compute the derivative ∇J.
  - Start at some point θ_0 and evaluate $\nabla J(\theta_0)$.
  - Make a step in the reverse direction of the gradient: $\theta_1 \leftarrow \theta_0 - \nabla J(\theta_0)$.
  - Repeat until J is sufficiently small.



Gradient Descent for UV Decomposition

- How to use GD to find U^* and V^* for UV decomposition:
- **Step 1:** Initialize U and V using SVD, treating the blank as 0.
  - Generally better than random initialization.
- **Step 2:** Update U and V to minimize the objective function J.
  - $U \leftarrow U - \eta \cdot \nabla_U J(R, U, V)$.
  - $V \leftarrow V - \eta \cdot \nabla_V J(R, U, V)$.
  - η is a hyperparameter (called a step size or a learning rate).

Gradient Descent for UV Decomposition

- Perform the update step on every entry independently.
  - Since U and V are matrices.
- For each entry at row x, column c of matrix U, we update:

$$u_{xc} = u_{xc} - \eta \cdot \nabla_{u_{xc}} J(R, U, V)$$

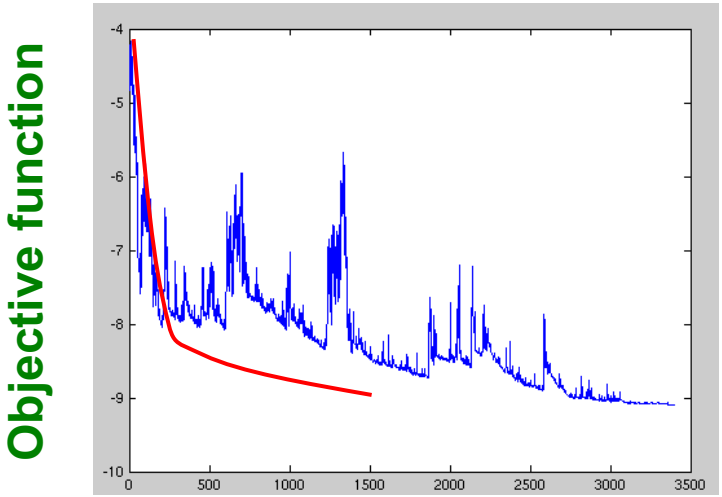
where $\nabla_{u_{xc}} J(R, U, V) = \sum_{i:(x,i)\in E} -2v_{ic}(r_{xi} - u_x^\top v_i) + 2\lambda_1 u_{xc}$.

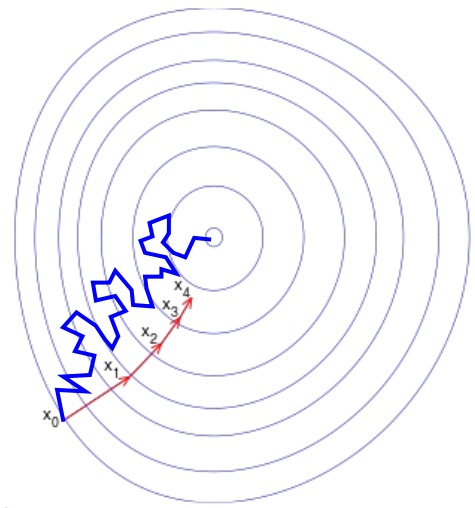
Stochastic Gradient Descent

- **Observation:** Gradient descent is too slow for large data.
- **Stochastic gradient descent (SGD):**
  - We know that $\nabla J(R, U, V) = \sum_{(x,i)\in E} \nabla J(r_{xi}, U, V)$.
  - Evaluate the gradient on each (not all) rating and make a step.
  - Needs more steps until convergence but each step is much faster.
  - **GD:** $U \leftarrow U - \eta \cdot \left[\sum_{(x,i)\in E} \nabla J(r_{xi}, U, V)\right]$.
  - **SGD:** $U \leftarrow U - \eta \cdot \nabla J(r_{xi}, U, V)$ for $(x, i) \in E$.

Convergence of SGD vs. GD

- GD improves the value of the objective function at every step.
- SGD improves the value but in a “noisy” way.
- GD takes fewer steps but each step takes much longer.





SGD for UV Decomposition

- How to use SGD to find U^* and V^* for UV decomposition:
- **Step 1:** Initialize U and V using SVD, treating the blank as 0.
  - Generally better than random initialization.
- **Step 2:** Iterate over the ratings until convergence:

```
for each r_xi ∈ E:
    ε_xi ← 2(r_xi − u_x^⊤ v_i)
    u_x ← u_x + η(ε_xi v_i − λ_1 u_x)
    v_i ← v_i + η(ε_xi u_x − λ_2 v_i)
```

SGD with Mini-batches

- In practice, people do not apply SGD for individual samples.
- Instead, they create (**mini-**)**batches** of several samples.
  - GD: 1 step using N samples.
  - True SGD: N steps using 1 sample.
  - (Batch) SGD: N/B steps using a batch of B samples.
- Leads to a better balance between speed and stability.
  - B is a hyperparameter to choose.

Outline

1. UV Decomposition
2. UV Decomposition as Machine Learning
3. **<u>Variants of UV Decomposition</u>**
4. Dealing with Implicit Feedback

Modeling Biases and Interactions

- There are **global effects (biases)** of users and movies.
  - Rating r_{xi} is not only about the **interaction** between x and i.
  - E.g., a critical reviewer x_1 vs. a generous person x_2.
  - E.g., The Godfather i_1 vs. some bad movie i_2 on Netflix.

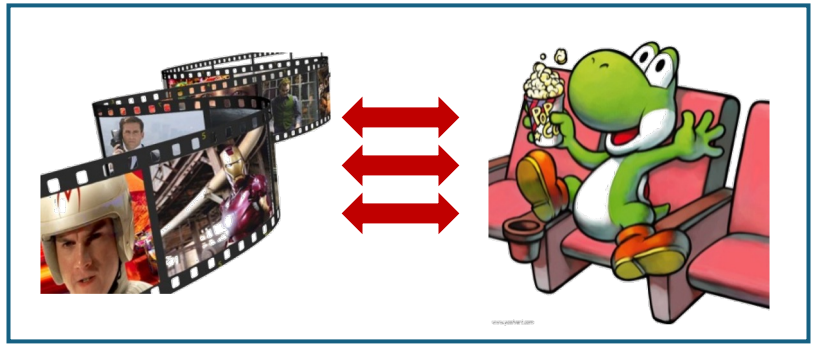
user bias

+

movie bias

+

user-movie interaction



Model with Biases

- Let's predict a rating as $\hat{r}_{xi} = \mu + b_x + b_i + u_x^\top v_i$.
  - μ is the overall mean rating.
  - b_x and b_i are biases for user x and movie i, respectively.
  - $u_x^\top v_i$ is their interaction modeled by factor matrices.
- **Example:**
  - Mean rating of training data is $\mu = 3.7$.
  - You are a critical reviewer: $b_x = -1$.
  - Star Wars is favored by many people: $b_i = +0.5$.
  - Final score is $3.7 - 1 + 0.5 + u_x^\top v_i$.

Fitting the New Model

- Update the parameters $\{U, V, B_{\text{user}}, B_{\text{item}}\}$ minimizing J':

$$J'(\cdot) = \sum_{x,i \in E} \left(r_{xi} - (\mu + b_x + b_i + u_x^\top v_i)\right)^2 + \text{regularizer}$$

  - There are 4 regularization hyperparameters: $\lambda_1, \lambda_2, \lambda_3, \lambda_4$.
  - μ is the simple average of ratings; it needs not to be learned.

Further Improvements

- Any idea for modifying the three components is okay to try:
  - Set θ of learnable parameters.
  - Objective function J to minimize.
  - (Optional) regularizer on θ with λ.
- SGD (or GD) will take care of finding the optimal parameters.
  - We believe in the power of gradient-based optimization!

Hyperparameter Search

- How can we search for optimal hyperparameters $\lambda_1, \lambda_2, \lambda_3, \lambda_4$?
- We pick the one with the best validation performance.
- **Grid search:** Create sets of values and try for each combination.
  - E.g., λ_1 in $\{0.001, 0.01, 0.1, 1, 10\}$.
- **Random search:** Try for each randomly selected combination.
  - E.g., λ_1 from $[0.001, 10]$ (but in a log scale).

Outline

1. UV Decomposition
2. UV Decomposition as Machine Learning
3. Variants of UV Decomposition
4. **<u>Dealing with Implicit Feedback</u>**

Dealing with Implicit Feedback

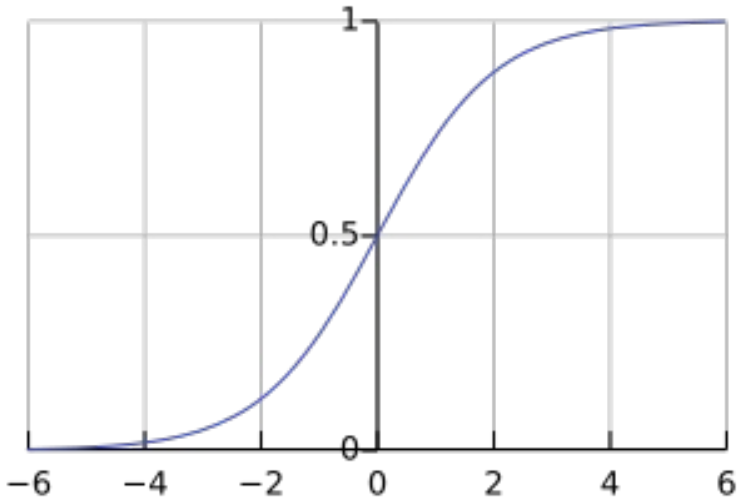
- What if the utility matrix R contains only implicit feedback?
  - Each entry is either 0 (not watched) or 1 (watched).
- Using the same model with RMSE J makes some limitations.
  - **Limitation 1:** Our prediction can be any number, i.e., $u_x^\top v_i > 1$.
  - **Limitation 2:** We may need a better gradient curve of J.
  - **Limitation 3:** Our loss J assumes 0 as a dislike, not "not watched."

Idea 1: Sigmoid Function

- We want to limit the predictions of our model to be $[0, 1]$.
- Simple solution is to use $\sigma(u_x^\top v_i)$ instead of $u_x^\top v_i$ as output.
- **Sigmoid function** σ is defined as follows:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

  - Maps $(-\infty, \infty)$ to $(0, 1)$ with $\sigma(0) = 0.5$.
  - Monotonically increasing for all ranges of x.



Implication of the Sigmoid

- **Without sigmoid:** Signals are mixed regardless of r_{xi}.
  - If $r_{xi} = 1$ and $u_x^\top v_i > 1$, the model is updated to decrease $u_x^\top v_i$.
  - If $r_{xi} = 1$ and $u_x^\top v_i < 1$, the model is updated to increase $u_x^\top v_i$.
  - If $r_{xi} = 0$ and $u_x^\top v_i > 0$, the model is updated to decrease $u_x^\top v_i$.
  - If $r_{xi} = 0$ and $u_x^\top v_i < 0$, the model is updated to increase $u_x^\top v_i$.
- **With sigmoid:** Signals are consistent with r_{xi}.
  - If $r_{xi} = 1$, the model is always updated to increase $u_x^\top v_i$.
  - If $r_{xi} = 0$, the model is always updated to decrease $u_x^\top v_i$.

Idea 2: Binary Cross Entropy

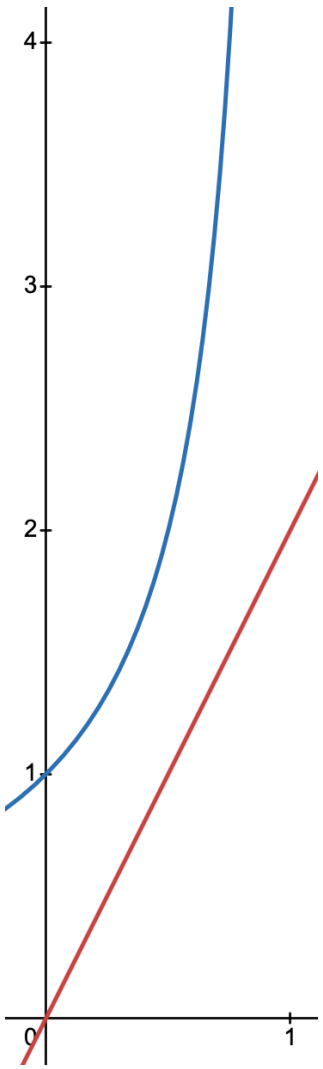
- RMSE is used mainly for continuous, unbounded targets.
- We may use **binary cross entropy (BCE)** instead of RMSE:

$$J_{\mathrm{BCE}}(r_{xi}, u_x, v_i) = -[r_{xi} \log \sigma(u_x^\top v_i) + (1 - r_{xi}) \log(1 - \sigma(u_x^\top v_i))]$$

  - If $r_{xi} = 1$, we minimize $-\log \sigma(u_x^\top v_i)$ by making $\sigma(u_x^\top v_i)$ to 1.
  - If $r_{xi} = 0$, we minimize $-\log(1 - \sigma(u_x^\top v_i))$ by making $\sigma(u_x^\top v_i)$ to 0.

Gradient Curves

- BCE has a gradient curve different from that of RMSE.
- Suppose that $r_{xi} = 0$ and $a = \sigma(u_x^\top v_i)$ for simplicity.
  - (RMSE) $\nabla_a \|a\|^2 = 2a$: Gradient increases linearly with a.
  - (BCE) $\nabla_a - \log(1-a) = 1/(1-a)$: Gradient increases rapidly.
- BCE makes the model focus more on **wrong samples**.
  - Not already accurate ones.



Idea 3: Ranking Losses

- Both RMSE and BCE assume 0 as "dislike," not "unknown."
  - Since they are computed for individual elements r_{xi}.
- **Idea:** Consider the task as **ranking**, not *elementwise prediction*.
  - Given a user x, suppose that $r_{xi} = 1$ while $r_{xj} = 0$.
  - Then, we can safely assume that user x likes i more than j.
    - If x really likes j, they would have watched it before i.
  - Let's update the model by comparing items, so that $u_{xi} > u_{xj}$.

Bayesian Personalized Ranking

- We may use the **Bayesian personalized ranking (BPR)** loss:

$$J_{\mathrm{BPR}}(U, V) = \sum_{(x,i,j)} -\log \sigma\left(u_x^\top v_i - u_x^\top v_j\right)$$

  - Item j is randomly selected from the **negative samples** $\{\, j \mid r_{xj} = 0 \,\}$.
- The sigmoid function σ enables all pairs to contribute fairly.
- How to sample negative pairs is also important (some ideas?).

Summary

1. UV Decomposition
   - Latent factor models
2. UV Decomposition as Machine Learning
   - Overfitting and regularization
   - Stochastic gradient descent
3. Variants of UV Decomposition
   - Modeling biases
4. UV Decomposition: Variants
   - BCE and BPR losses